

Experiment-01: 8-Puzzle Problem using A Search Algorithm

Aim

To solve the **8-Puzzle Problem** using the **A* Search Algorithm** with the **Manhattan Distance heuristic** to find the shortest path from the initial state to the goal state.

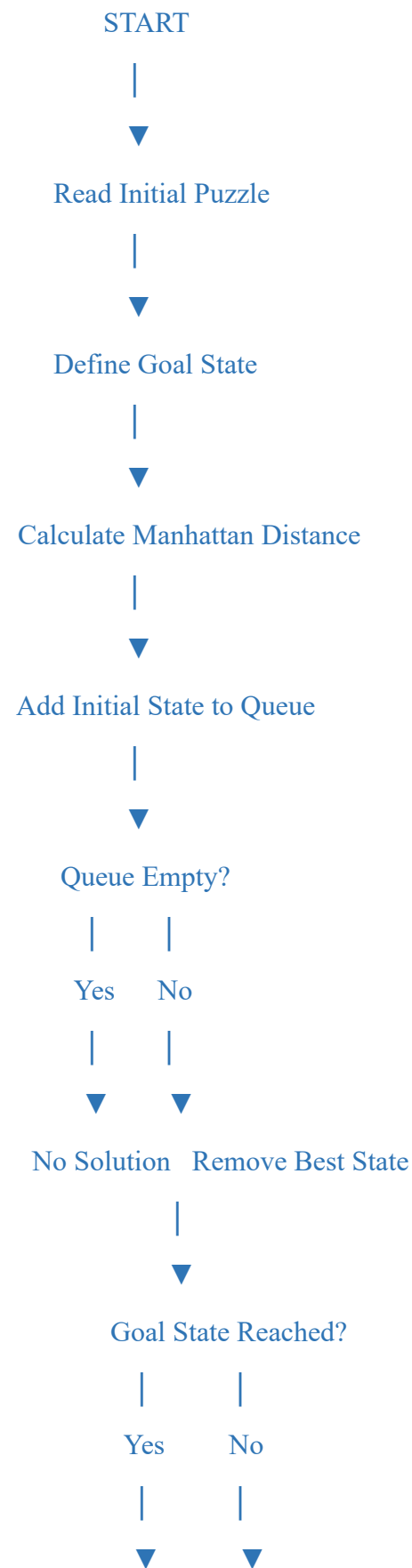
Objective

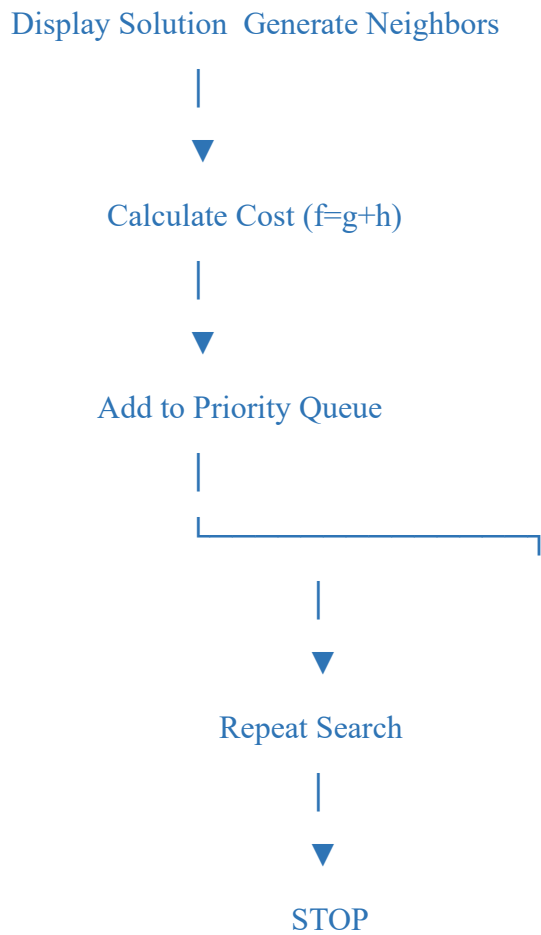
- To understand the working of the A* Search Algorithm.
- To find the optimal sequence of moves for the 8-puzzle.
- To use the Manhattan Distance heuristic to improve search efficiency.
- To reach the goal state with the minimum number of moves.

Algorithm

1. Start.
2. Read the initial puzzle configuration.
3. Define the goal state.
4. Calculate the heuristic value (Manhattan Distance).
5. Insert the initial state into the priority queue.
6. Repeat until the queue becomes empty:
 - Remove the state with the lowest cost.
 - If it is the goal state, display the solution.
 - Otherwise, generate all valid neighboring states.
 - Calculate the cost for each neighbor.
 - Insert unexplored neighbors into the priority queue.
7. If no solution exists, display a message.
8. Stop.

Flowchart





Python Code

```
import heapq
```

```
GOAL = [[1,2,3],  
        [4,5,6],  
        [7,8,0]]
```

```
def find_blank(state):  
    for i in range(3):  
        for j in range(3):  
            if state[i][j]==0:  
                return i,j
```

```

def heuristic(state):
    distance=0
    for i in range(3):
        for j in range(3):
            value=state[i][j]
            if value!=0:
                gx=(value-1)//3
                gy=(value-1)%3
                distance+=abs(i-gx)+abs(j-gy)
    return distance

def get_neighbors(state):
    neighbors=[]
    x,y=find_blank(state)

    directions=[(-1,0),(1,0),(0,-1),(0,1)]

    for dx,dy in directions:
        nx=x+dx
        ny=y+dy

        if 0<=nx<3 and 0<=ny<3:
            new=[row[:] for row in state]
            new[x][y],new[nx][ny]=new[nx][ny],new[x][y]
            neighbors.append(new)

    return neighbors

def to_tuple(state):

```

```
return tuple(tuple(row) for row in state)
```

```
def solve(start):
```

```
    pq=[]
```

```
    heapq.heappush(pq,(heuristic(start),0,start,[]))
```

```
    visited=set()
```

```
    while pq:
```

```
        f,g,current,path=heapq.heappop(pq)
```

```
        if current==GOAL:
```

```
            return path+[current]
```

```
        t=to_tuple(current)
```

```
        if t in visited:
```

```
            continue
```

```
        visited.add(t)
```

```
        for neighbor in get_neighbors(current):
```

```
            if to_tuple(neighbor) not in visited:
```

```
                heapq.heappush(
```

```
                    pq,
```

```
                    (g+1+heuristic(neighbor),
```

```
                    g+1,
```

```
                    neighbor,
```

```
                    path+[current])
```

```
            )
```

```
        return None

start=[[1,2,3],
       [4,0,6],
       [7,5,8]]
solution=solve(start)
if solution:
    print("Solution Found")
    for step in solution:
        for row in step:
            print(row)
        print()
else:
    print("No Solution")
```

Sample Input

Initial State

1 2 3

4 0 6

7 5 8

Goal State

1 2 3

4 5 6

7 8 0

Sample Output

Solution Found

[1, 2, 3]

[4, 0, 6]

[7, 5, 8]

[1, 2, 3]

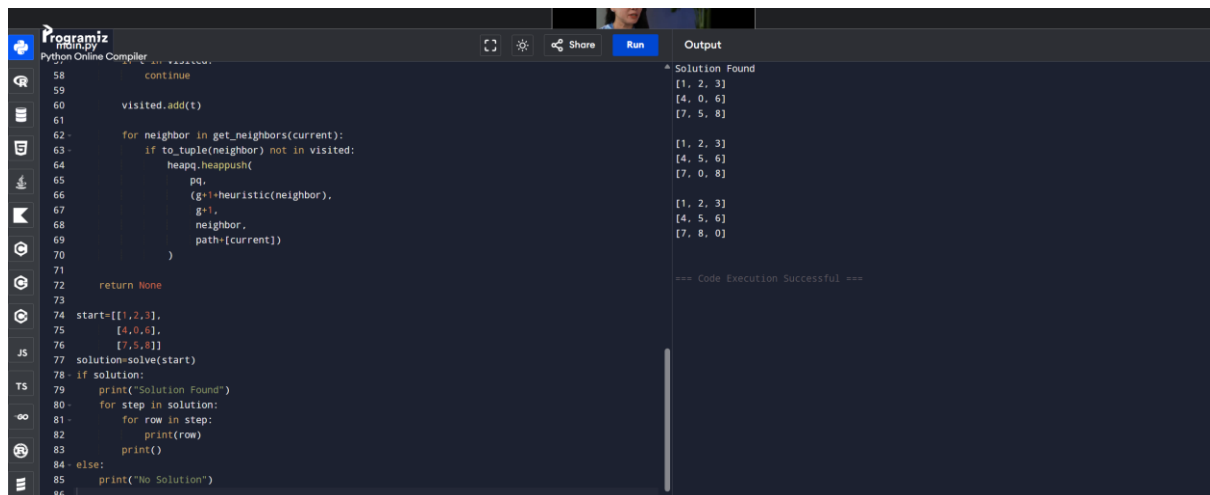
[4, 5, 6]

[7, 0, 8]

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]



The screenshot shows a Python Online Compiler interface. The code implements an A* search algorithm for the 8-puzzle problem. It defines a heuristic function, a get_neighbors function, and a solve function. The solve function starts with an initial state and explores its neighbors until it finds the goal state. The output shows the solution found: [1, 2, 3], [4, 0, 6], [7, 5, 8], [1, 2, 3], [4, 5, 6], [7, 0, 8], [1, 2, 3], [4, 5, 6], [7, 8, 0]. The code execution is successful.

```
58         continue
59
60         visited.add(t)
61
62         for neighbor in get_neighbors(current):
63             if to_tuple(neighbor) not in visited:
64                 heapq.heappush(
65                     pq,
66                     (g+1+heuristic(neighbor),
67                     g+1,
68                     neighbor,
69                     path+[current]))
70
71
72     return None
73
74 start=[[1,2,3],
75        [4,0,6],
76        [7,5,8]]
77 solution=solve(start)
78 if solution:
79     print("Solution Found")
80     for step in solution:
81         for row in step:
82             print(row)
83         print()
84 else:
85     print("No Solution")
86
```

Output

```
^ Solution Found
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
=== Code Execution Successful ===
```

Result

The 8-Puzzle problem was successfully solved using the **A* Search Algorithm**. The algorithm generated the optimal sequence of moves and reached the goal state with the minimum path cost.